



UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

DIPARTIMENTO DI INGEGNERIA
DELL'INFORMAZIONE

Corso di Laurea in Ingegneria Informatica

Relazione finale

PROGETTAZIONE E SVILUPPO DI UN
SISTEMA RAG
PER L'ACCESSO INTELLIGENTE ALLA
CONOSCENZA AZIENDALE

Relatore:

Chiar.mo Prof. Serina Ivan

Chiar.mo Prof. Gerevini Alfonso Emilio

Laureando:

Massolari Mattia

Matricola n.:

740458

Anno Accademico 2024/2025

Indice

1	Introduzione	2
2	Stato dell'arte	4
2.1	Intelligenza Artificiale AI	4
2.2	Machine Learning	5
2.3	Deep Learning	6
2.4	Natural Language Processing	6
2.5	Transformers	7
2.6	Large Language Models	9
2.7	Retrieval Augmented Generation	9
2.7.1	Funzionamento della RAG	10
3	Metodologia	14
3.1	Implementazione RAG	14
3.1.1	LangChain	15
3.1.2	Analisi e preparazione dei dati	17
3.1.3	Splitting e Embedding	20
3.1.4	Prompt and query	23
4	Analisi Sperimentale	27
4.1	Modalità di test	27
4.1.1	Domande e risposte	27
4.1.2	Metodologia Similarity Search	28
4.1.3	Metodologia Max Marginal Relevance	30
4.2	Analisi dei risultati	30
4.2.1	Confronto dei modelli LLM	33
5	Conclusione	36
	Bibliografia	38

1 Introduzione

Negli ultimi anni il processo tecnologico ha trasformato la vita delle persone e delle aziende che hanno dovuto adattarsi al rapido cambiamento e concepire soluzioni innovative. Un esempio significativo è rappresentato da Mega Italia Media, fondata nel 1988 e con sede a Castel Mella (BS), tra i principali operatori italiani nella formazione aziendale digitale, con particolare attenzione alla salute e sicurezza sul lavoro. L'azienda sviluppa corsi eLearning e blended, fornisce una piattaforma Learning Management System (LMS) per la gestione e il monitoraggio della didattica e offre servizi di consulenza e contenuti multimediali aggiornati, con l'obiettivo di supportare imprese e professionisti attraverso soluzioni digitali che semplificano la formazione e garantiscono la conformità normativa.

Nella piattaforma DynDevice sono state create diverse funzionalità che possono essere utilizzate per migliorare l'efficienza operativa e l'organizzazione interna. Ecco che tra le altre, emerge una pagina di progress dove il personale aziendale può formulare domande, condividere documenti o segnalazioni e ricevere risposte dai colleghi o dai responsabili. In alcuni casi però capita che alcune domande siano già state proposte o che ci siano risposte già affrontate precedentemente e in questo modo l'utente dovrebbe stare a cercare manualmente tra le svariate conversazioni per individuare l'informazione desiderata. Per superare questa criticità, il presente elaborato propone una soluzione intelligente attraverso la progettazione e lo sviluppo di un sistema Retrieval Augmented Generation (RAG), concepito per dare risposte mirate all'utente partendo da una semplice domanda, evitando la ricerca manuale tra i contenuti presenti. Il sistema RAG combina la capacità di recupero di informazioni strutturate con la generazione linguistica dei LLM, permettendo risposte contestualizzate e pertinenti alle domande degli utenti.

Nella fase di sviluppo è stato utilizzato Python come linguaggio di programmazione in quanto ha facilitato la gestione dei dati e reso disponibili diverse librerie per l'implementazione dei modelli linguistici. Tra i modelli di OpenAI utilizzati risaltano GPT-3.5-Turbo, GPT-4, GPT-4.1, e non solo, anche Gemini-2.5-flash e Mistral-Tiny, rispettivamente di Google e Mistral AI, con l'obiettivo di individuare la soluzione più efficace in termini di affidabilità e accuratezza.

La tesi offre quindi un percorso che unisce l'analisi teorica dei dati e la progettazione applicativa: innanzitutto verranno introdotti i concetti chiave dell'intelligenza artificiale, con riferimento particolare a Natural Language Processing (NLP) molto importante per RAG fino ad arrivare ai Large Language Models (LLM) chiarendone potenzialità, limi-

tazioni e altre caratteristiche fondamentali. In seguito verrà affrontato il tema RAG nel suo funzionamento, metodologie tecniche e strumenti utilizzati. Infine verranno proposti i risultati ottenuti, in particolare con precisione e accuratezza delle risposte, per riuscire a garantire un sistema preciso e di qualità utilizzabile in un contesto aziendale.

2 Stato dell'arte

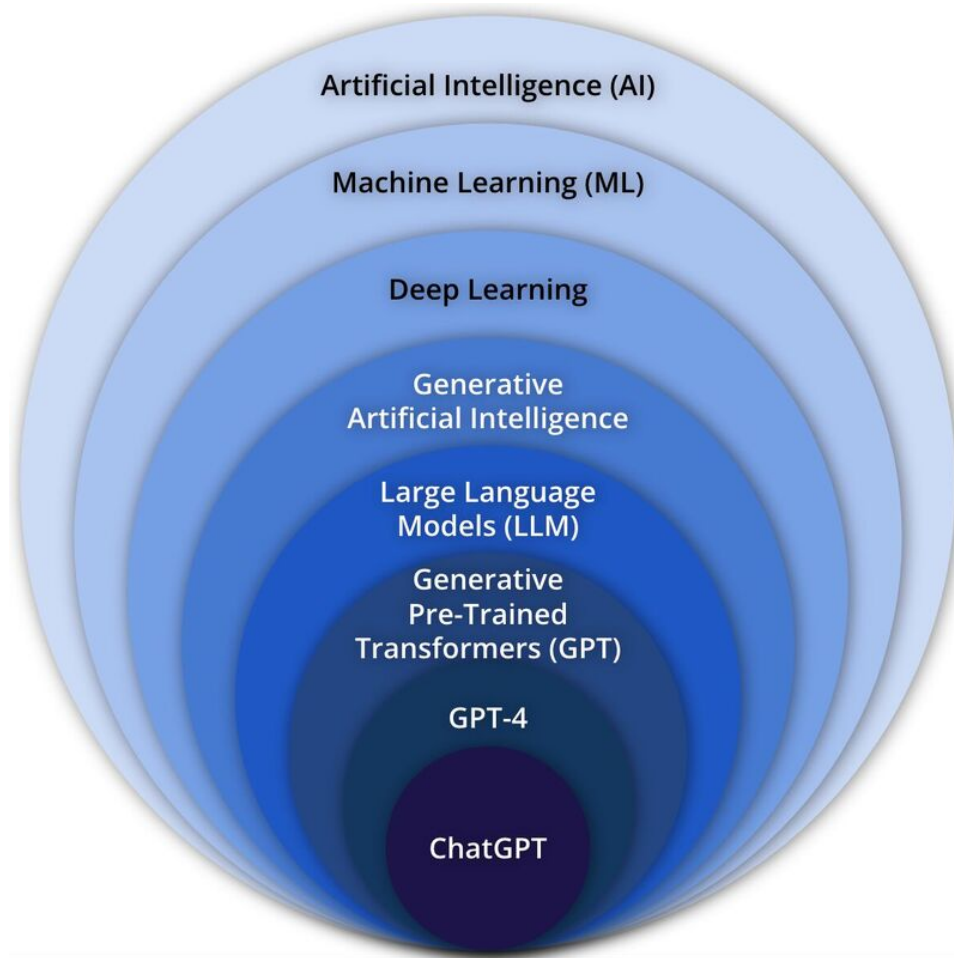


Figura 1: Il cuore del mondo AI.

2.1 Intelligenza Artificiale AI

Prima di parlare del sistema RAG, bisogna affrontare il concetto da cui tutto ha avuto inizio: Intelligenza Artificiale. Artificial Intelligence (AI) consente alle macchine di adeguarsi, apprendere nuove informazioni e di svolgere compiti simili a quelli dell'uomo. Il termine venne coniato nel 1956 per la prima volta, ma col passare degli anni la sua potenza è cresciuta a dismisura, con algoritmi avanzati, archiviazione e volume di dati impressionanti e potenza di calcolo elevatissima. Inizialmente le ricerche si concentravano sulla creazione di programmi capaci di limitare il ragionamento umano e le deduzioni logiche, poi diventò importante per facilitare progetti consistenti come fece la DARPA nella mappatura delle strade. E infine come la conosciamo noi oggi ovvero attraverso assistenti personali intelligenti come Alexa, Siri o Cortana.

L'intelligenza artificiale si è evoluta profondamente per fornire vantaggi specifici in ogni settore ed è sempre in continuo aggiornamento. Ci possono essere diversi esempi dei settori nei quali AI ha dato il suo contributo: nella sanità per letture mediche e radiografiche personalizzate, ricordando di prendere una pillola, fare esercizio fisico o mangiare in modo più sano; nel settore manifatturiero per l'analisi di dati aziendali; nel settore di retail, shopping virtuale, per offrire consigli, presentare diverse opzioni di acquisto o per aiutare nella gestione delle risorse in magazzino; nel settore delle banche le tecniche AI possono essere utilizzate per evitare transazioni fraudolente e automatizzare le attività più gravose di organizzazione dei dati [11].

Ma perché l'intelligenza artificiale oggi è così importante? L'AI automatizza apprendimento continuo, aggiunge intelligenza, si adatta attraverso algoritmi di apprendimento progressivo, analizza dati sempre più in profondità. Tutte caratteristiche che rendono AI sempre forte e affidabile, un ottimo braccio destro per l'essere umano.

Ma come funziona AI? AI funziona combinando grandi quantità di dati con elaborazione di algoritmi consentendo al software di imparare automaticamente dai modelli o dalle caratteristiche dei dati. AI offre diverse teorie, metodi, tecnologie: Machine Learning, Deep Learning o Natural Language Processing, campi differenti che supportano l'intelligenza artificiale [11].

2.2 Machine Learning

Il machine learning è un metodo di analisi che automatizza la costruzione di modelli analitici. È una branca dell'Intelligenza Artificiale e si basa sull'idea che i sistemi possono imparare dai dati, identificare modelli autonomamente e prendere decisioni con un intervento umano ridotto al minimo.

La costruzione di modelli precisi permette alle aziende di identificare nuove opportunità di profitto e di evitare rischi: a questo proposito serve che ci siano dei dati ben strutturati, algoritmi avanzati, processi di automazione iterativi, scalabilità e capacità di apprendimento d'insieme. Esistono tre metodi principali per il machine learning [12, 11]:

- **Apprendimento supervisionato** – l'algoritmo apprende a partire da dati etichettati (input e output noti), come nei modelli di classificazione e regressione.
- **Apprendimento non supervisionato** – i dati non hanno etichette predefinite e l'algoritmo cerca autonomamente strutture o raggruppamenti (clustering, riduzione di dimensionalità).

2.3 Deep Learning

Il deep learning è un ramo del machine learning che addestra i computer a svolgere compiti simili a quelli umani come il riconoscimento vocale, identificazione di immagini o elaborazione di previsioni. Riesce a migliorare la capacità di classificare, riconoscere, rilevare e descrivere attraverso i dati. Le architetture più comuni nel deep learning includono:

- **Reti neurali convoluzionali (CNN)** – particolarmente efficaci nell’elaborazione di immagini e video, grazie alla capacità di rilevare automaticamente pattern spaziali.
- **Reti neurali ricorrenti (RNN)** – progettate per gestire dati sequenziali come testo, audio o serie temporali, mantenendo una “memoria” degli stati precedenti.
- **Transformers** – architetture basate su meccanismi di attenzione, utilizzate principalmente nell’elaborazione del linguaggio naturale e alla base di molti modelli linguistici di ultima generazione.

Questa tecnologia ha portato a risultati molto utili anche per la generazione di contenuti testuali e multimediali [11, 13].

2.4 Natural Language Processing

Il natural language processing è una branca dell’intelligenza artificiale che aiuta il computer a capire, interpretare e manipolare il linguaggio umano. Il NLP punta a colmare il vuoto che è presente tra comunicazione umana e comprensione del computer.

Il NLP ha un ruolo molto importante perché oltre ad analizzare e comprendere il linguaggio umano che è straordinariamente complesso e diversificato, contribuisce a risolvere ambiguità linguistiche fornendo ai dati una struttura numerica comprensibile per la macchina. Grazie al machine e deep learning l’apprendimento è molto utilizzato per modellare il linguaggio umano, molte volte l’AI deve capire da errori di ortografia, interpretare punteggiatura o abbreviazioni. Le attività del NLP potrebbero essere intese come l’analisi logica e grammaticale della lingua italiana: il linguaggio viene scomposto in piccole parti elementari, cercando di comprendere le relazioni che esistono ed esaminando la loro combinazione per creare significato.

Per attività più complesse NLP può essere utilizzato per categorizzare dei contenuti, modellare dati, estrarre informazioni contestuali, eseguire sentiment analysis, trascrivere

una conversazione in testo e viceversa, riassumere documenti e tradurre automaticamente. L'obiettivo finale rimane quello di estrarre dall'input grezzo informazioni strutturate e di valore affinché possa essere compreso e utilizzato dalla macchina [11].

Una branca del NLP è il Natural Language Generation (NLG) che si concentra sulla produzione di linguaggio simile a quello umano prelevando i dati in qualunque forma, strutturata o non strutturata. L'obiettivo è quello di migliorare l'interazione uomo-macchina generando testo coerente e rilevante. Tuttavia, questo risultato non è semplice da ottenere perché modelli come GPT o BERT possono incorrere nel fenomeno delle allucinazioni: in determinate occasioni forniscono risposte inesatte, inventando informazioni non presenti o riportando dati errati. Questa limitazione ha portato all'esplorazione di modelli ibridi che combinano le loro abilità per cercare di generare il più possibile degli output coerenti: il fattore principale è la combinazione tra retrieval and generation che in modelli come REALM ha rappresentato la giusta chiave per allineare il contenuto e ottenere un miglioramento sia nel recupero delle informazioni sia nella generazione dell'output. REALM è un modello che integra il recupero dei documenti già nella fase di pre-training. In questo modo il retriever e il generatore vengono addestrati insieme, consentendo al sistema di selezionare informazioni utili per la generazione, come il question answering o il riassunto di testi. Questa architettura dimostra dei sensibili miglioramenti per l'accuratezza delle risposte rispetto alle metodologie tradizionali [4].

2.5 Transformers

I modelli Transformer sono alla base di molti sistemi di NLP e di LLM avanzati, incluso il sistema RAG. Una delle innovazioni più importanti dei transformer è che sfruttano il meccanismo di self-attention che consente di attribuire dei pesi alle parole di una sequenza permettendo di catturare relazioni semantiche anche tra termini distanti.

Esistono diverse configurazioni di modelli Transformer:

- **Solo encoder** (es. *BERT*) – usati principalmente per compiti di comprensione del testo, come classificazione, analisi del sentiment o estrazione di informazioni.
- **Solo decoder** (es. *GPT*) – ottimizzati per la generazione di testo in linguaggio naturale, come conversazioni o scrittura creativa.
- **Encoder-decoder** (es. *BART*, *T5*) – impiegati per compiti complessi come traduzione automatica, riassunto o question answering avanzato [1].

Tutti questi modelli sono stati addestrati come modelli linguistici attraverso apprendimento auto-supervisionato, che significa interagire solamente con input senza bisogno dell'intervento umano per etichettare i dati. Tuttavia per compiti precisi e pratici, serve un affinamento supervisionato (fine-tuning), che permette al modello di svolgere attività con maggior precisione. Questo si traduce in maggiori quantità di dati che il modello deve sostenere rilevando inevitabilmente elevati costi in termini di risorse e tempo come si può notare dalla Figura 2.

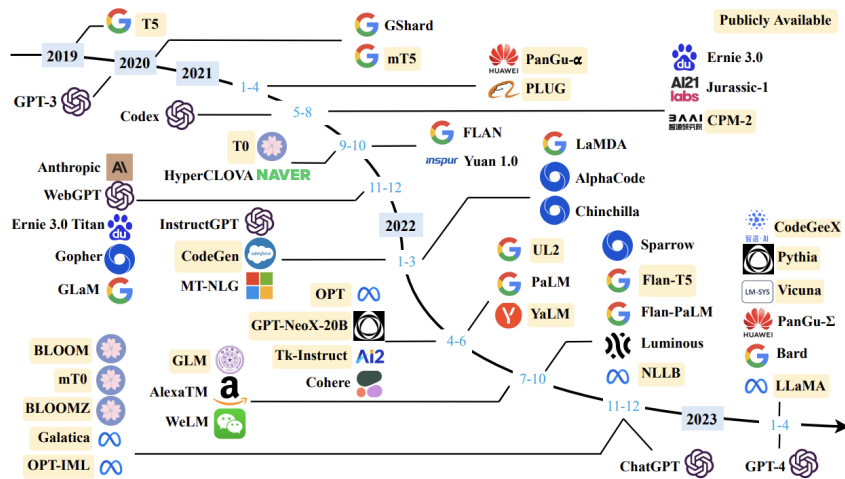


Figura 2: Cronologia degli LLM sviluppati negli ultimi anni, con numero di parametri superiore ai 10 miliardi. I modelli con checkpoint pubblicamente disponibile sono evidenziati in giallo.

La condivisione di modelli linguistici è fondamentale: lavorare a partire da modelli già addestrati riduce i costi informatici complessivi e l'impatto ambientale.

Il pre-addestramento è l'addestramento di un modello da zero, prende in considerazione dunque una mole di dati molto grande e per questo motivo può metterci diverse settimane oltre a dover possedere un ampio archivio. Il fine-tuning invece è un addestramento supervisionato molto efficace che ha luogo solo dopo che il modello è stato pre-addestrato e serve per affinare un modello in base al compito che deve svolgere. Il fine-tuning di un modello ha quindi costi ridotti in termini di dati, finanze e impatto ambientale. Questo processo permette anche di ottenere risultati migliori di un addestramento da zero, motivo per cui bisognerebbe sempre partire da un modello pre-addestrato e affinarlo [17].

2.6 Large Language Models

Large language models (LLM) sono modelli linguistici avanzati in grado di comprendere e generare linguaggio naturale. Hanno la capacità di dedurre dal contesto, generare risposte coerenti e contestualmente pertinenti, tradurre in diverse lingue, riassumere il testo, rispondere a domande e assistere nella generazione di codice. Gli LLM acquisiscono questa capacità adoperando enormi quantità di dati per apprendere miliardi di parametri durante l'addestramento. Gli LLM sono in larga parte Transformer e sono pre-addestrati attraverso l'apprendimento auto-supervisionato o l'apprendimento semisupervisionato. Questi modelli sono pre-addestrati predicendo ripetutamente la parola o il simbolo immediatamente successivi nella sequenza. Nelle applicazioni pratiche, invece di ricorrere sempre al fine-tuning supervisionato, si utilizza il prompt engineering, ovvero la costruzione di istruzioni mirate, che consente di indirizzare il comportamento del modello senza doverlo riaddestrare.

Un fattore chiave nel funzionamento degli LLM è il modo in cui si rappresentano le parole. Un aspetto fondamentale nel funzionamento degli LLM è la rappresentazione delle parole: esse vengono mappate in vettori multidimensionali, in cui termini con significato simile siano vicini tra loro nello spazio vettoriale.

In questo modo i Transformers possono elaborare preventivamente il testo tramite l'encoder, comprendendo il contesto di parole e frasi con significati simili, e possono generare output coerenti tramite il decoder.

Esempi noti di modelli linguistici di grandi dimensioni sono i modelli GPT di OpenAI (ad esempio GPT-3, oppure GPT-3.5-turbo e GPT-4), PaLM di Google (usato in Gemini) e LLaMa di Meta [15, 16, 18].

2.7 Retrieval Augmented Generation

La Retrieval Augmented Generation è un approccio che estende le capacità già avanzate degli LLM senza necessità di riaddestrare il modello per fare riferimenti a fonti esterne autorevoli prima di generare una risposta. Questo migliora la coerenza, l'accuratezza e l'utilità delle risposte [2]. La RAG integra due componenti:

- **Meccanismo di recupero** che estrae informazioni rilevanti o dati da fonti esterne;
- **Modello di generazione** che processa le informazioni recuperate per produrre risposte simili a quelle umane.

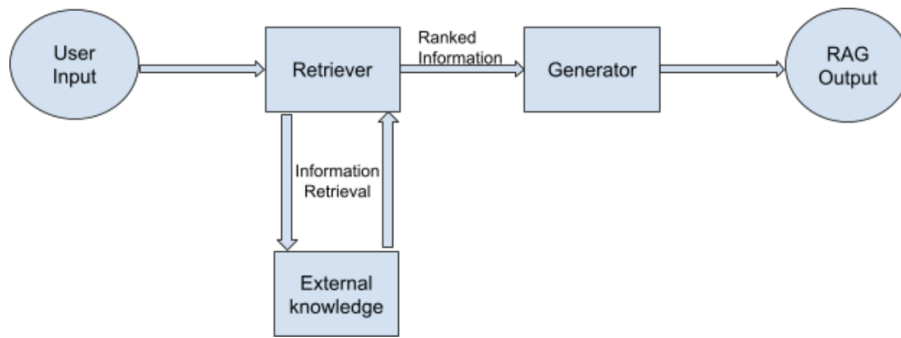


Figura 3: Sistema RAG con i suoi componenti principali

Una sfida nella generazione di testo è quella di cercare di evitare le allucinazioni, ovvero la generazione di informazioni false o fuorvianti. Infatti i modelli LLM tradizionali necessitano spesso di un riaddestramento per migliorare la loro accuratezza e affidabilità. L'architettura RAG punta a ridurre le allucinazioni recuperando informazioni e generando del testo sempre aggiornato, preciso e rilevante. Per questo motivo la RAG è davvero abile e flessibile in compiti come risposte a domande, traduzione, generazione di testo e completamento di frasi [2]. Inoltre, la RAG offre altri vantaggi:

- **Implementazione conveniente:** consente di introdurre nuovi dati senza riaddestrare il modello, mantenendo il sistema aggiornato.
- **Informazioni attuali:** il modello accede a dati live da fonti aggiornate, migliorando accuratezza e pertinenza.
- **Maggiore fiducia degli utenti:** le risposte possono includere riferimenti a fonti verificabili.
- **Maggiore controllo per gli sviluppatori:** è possibile monitorare e gestire le fonti, limitando informazioni sensibili e correggendo errori.

In questo modo le organizzazioni possono implementare la tecnologia IA generativa con maggiore sicurezza [2].

2.7.1 Funzionamento della RAG

A differenza di un modello LLM tradizionale che genera un output solo dai dati di training, la RAG integra un componente di recupero delle informazioni. Per costruire un sistema RAG è necessario avere una base di dati iniziali, fare **Indexing** dei dati e creare

il database vettoriale. Il modello RAG può essere utilizzato per rispondere a domande specifiche, abbiamo tre fasi: **Indexing**, **Retrieval** e **Generation**.

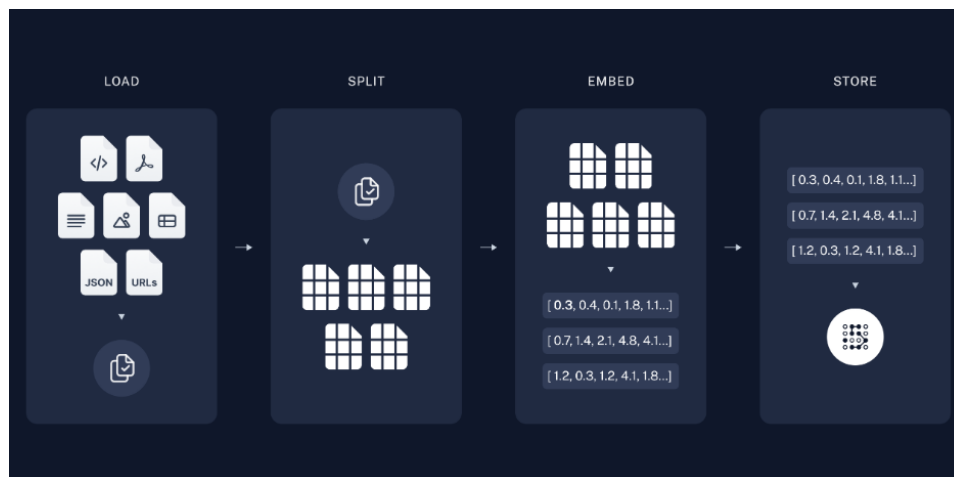


Figura 4: Indexing.

L'**Indexing** è il processo che riguarda l'estrazione e la pulizia dei dati che possono essere in diversi formati come PDF, HTML, Word o altro. Questi dati vengono convertiti in un testo semplice e uniforme per poi essere suddiviso in diverse porzioni più piccole, chiamate *chunk*. Questi chunk vengono indicizzati in rappresentazioni vettoriali mediante un modello di embedding e inseriti in un database vettoriale. Come possiamo notare dalla Figura 4, l'Indexing è divisa in 4 fasi principali:

1. **Load:** caricamento dei documenti che fungono da dati del sistema
2. **Split:** i documenti vengono sezionati in piccoli porzioni di testo detti chunk. Passaggio molto utile per i modelli che devono andare a cercare informazioni in sezioni di testo ridotto rispetto ad interi documenti.
3. **Embed:** i chunk vengono convertiti in vettori numerici più facilmente interpretabili dal modello.
4. **Store:** i vettori numerici dei vari chunk vengono salvati in un archivio, un database vettoriale, per essere poi recuperati dal modello nella ricerca [19].

Il **Retrieval** è il processo per cui viene effettuata una ricerca nel database vettoriale per trovare i chunk più rilevanti in base alla query dell'utente. Il sistema rileva e dà priorità ai top K chunk più pertinenti che dimostrano una più alta similarità con la

query. Questi chunk sono poi utilizzati per espandere il contenuto del prompt e fornire contesto. La **Generation** è il processo finale in cui il modello LLM genera una risposta. La query dell'utente viene combinata con i chunk recuperati che vengono sintetizzati in un prompt coerente. LLM può generare la risposta in base alla propria conoscenza parametrica oppure limitare le risposte al contesto fornito dai chunk recuperati. In alcuni casi anche lo storico della conversazione può essere utilizzato per migliorare la coerenza e la pertinenza delle risposte generate [3].

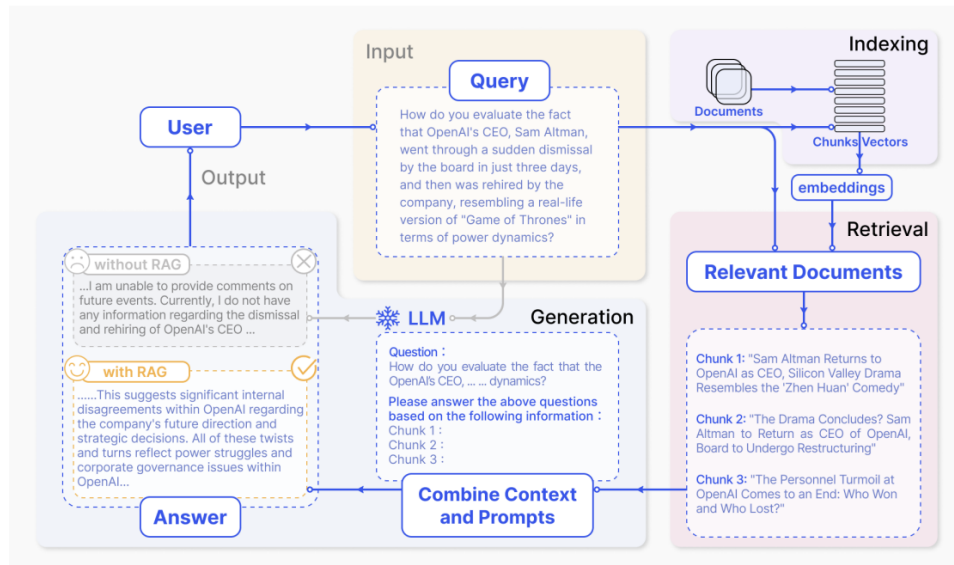


Figura 5: Istanza di RAG per un task di Question Answering.

Il processo, descritto in Figura 5 è il seguente:

1. L'utente immette un input che corrisponde alla **Query**, la quale può essere una domanda o una richiesta specifica;
2. L'**Indexing** permette di frammentare i dati iniziali in chunk, i quali vengono indicizzati nel database vettoriale;
3. Il **Retrieval** che recupera dal database vettoriale i chunk più rilevanti rispetto alla query proposta dall'utente attraverso una funzione di similarità;
4. Il contesto recuperato nella fase precedente viene combinato con il prompt per LLM in modo che il modello abbia tutto il necessario per fornire un'accurata risposta;
5. Ultima fase corrisponde alla generazione di un output da parte dell'LLM che sfrutta il prompt fornitogli per generare una risposta chiara ed efficiente [2].

L'evoluzione dai modelli LLM tradizionali alla RAG ha permesso di superare limiti come aggiornamento delle informazioni e allucinazioni, aprendo la strada a sistemi più affidabili e controllabili. Nei capitoli successivi verrà illustrata l'implementazione pratica di un sistema RAG, con dettagli sull'architettura, le tecnologie impiegate e i risultati ottenuti.

3 Metodologia

In questo capitolo viene illustrata la metodologia adottata per la progettazione e la realizzazione del sistema RAG. Verranno descritte le fasi principali del flusso di lavoro, le operazioni di caricamento e indicizzazione dei documenti, le tecniche di recupero e generazione delle risposte. L'obiettivo è fornire una panoramica chiara e strutturata delle scelte progettuali e delle motivazioni alla base di ciascun passaggio.

3.1 Implementazione RAG

In questa sezione viene descritto come è stata implementata l'architettura RAG. Nello svolgimento del progetto l'elemento centrale è stato l'uso dell'API Assistant di OpenAI e di Google che ha permesso di integrare LLM per rispondere a domande basate su documenti forniti. Questa tecnologia facilita l'elaborazione del linguaggio naturale e migliora la qualità delle risposte generate, ottimizzando la gestione delle informazioni. Sono stati utilizzati diversi modelli di GPT (Generative Pre-trained Transformer) forniti da OpenAI:

- **GPT-4.1**: versione avanzata della famiglia GPT-4, ottimizzata per una maggiore accuratezza, capacità di ragionamento e gestione di contesti lunghi. È adatto a compiti complessi di comprensione e generazione di testo, fornendo risposte coerenti e ben argomentate;
- **GPT-4-turbo**: variante più veloce ed economica di GPT-4, progettata per ridurre la latenza e i costi mantenendo una qualità di risposta elevata. Ideale per applicazioni che richiedono molte richieste API in tempi ridotti;
- **GPT-4**: modello progettato per offrire un'elevata qualità di ragionamento, comprensione del contesto e generazione di testo naturale. È particolarmente adatto a compiti complessi che richiedono risposte dettagliate;
- **GPT-3.5-turbo**: modello più leggero rispetto alla serie GPT-4, con buone prestazioni in compiti di comprensione e generazione di testo. Offre un compromesso tra qualità e velocità, utile in scenari dove i costi e le prestazioni devono essere bilanciati.

Tra gli altri disponibili sono stati utilizzati anche modelli di Google e di Mistral AI:

- **Gemini-2.5-flash**: modello di Google della famiglia Gemini, ottimizzato per risposte rapide e a bassa latenza. È progettato per applicazioni interattive e di ricerca veloce di informazioni, sacrificando parzialmente la profondità di ragionamento a favore della velocità;
- **Mistral-Tiny**: modello open-source sviluppato da Mistral AI, leggero e ottimizzato per l'elaborazione veloce di testo. Adatto a compiti che richiedono tempi di risposta rapidi e consumi computazionali ridotti, pur mantenendo una buona coerenza nelle risposte.

Per ogni modello potremo osservare le performance rispetto a due tipologie di richieste: domande a risposta chiusa e domande a risposta aperta.

3.1.1 LangChain

Per la costruzione del sistema RAG utilizzato nel progetto è stato impiegato **LangChain** [19], una libreria Python che facilita lo sviluppo di applicazioni basate su modelli di linguaggio integrati con retrieval e gestione del contesto. LangChain fornisce componenti per collegare i modelli LLM a fonti di dati esterne, come database vettoriali o documenti testuali, e per orchestrare l'intero processo di **Retrieval and Generation** in modo flessibile.

La libreria supporta concetti fondamentali come **Chains**, che permettono di concatenare più passaggi di elaborazione, e **Agents**, che consentono a un modello di decidere quali strumenti utilizzare in base alla query dell'utente. Inoltre, LangChain mette a disposizione strumenti per il caricamento e la gestione dei documenti, la creazione di embedding e la gestione dei **Vector Stores**, come FAISS, per il recupero dei chunk più rilevanti.

Nel progetto, come si può vedere dal Listato 1, LangChain è stato utilizzato per: la segmentazione dei documenti in chunk, la generazione degli embedding e la creazione dell'indice FAISS, fino alla ricerca dei chunk più pertinenti e alla costruzione del prompt finale da inviare al modello LLM. In questo modo, si separa chiaramente la logica del retrieval dalla generazione della risposta.

```

1 # JSON file cleaned
2 grouped = msg.groupby("progress_id")
3 dataset = []
4 for progress_id, group in grouped:
5     dataset.append({
6         "progress_id": progress_id,
```

```

7         "subject": meta["subject"],
8         "description": meta["description"],
9         "created_at": meta["created_at"],
10        "closed_at": meta["closed_at"],
11        "updated_at": meta["updated_at"],
12        "author": meta["author"],
13        "message_count": len(messages),
14        "total_char_length": total_length,
15        "distinct_authors": len(unique_authors),
16        "messages": messages
17    })
18
19    dataset = [p for p in dataset if p["message_count"] > 0]
20
21    with open(JSON_OUTPUT, "w", encoding="utf-8", errors="replace") as f:
22        json.dump(dataset, f, ensure_ascii=False, indent=2)
23
24    # Splitting documents
25    splitter = RecursiveCharacterTextSplitter(chunk_size=500,
26        chunk_overlap=50)
27    chunks = splitter.split_documents(docs)
28    chunks = [chunk for chunk in chunks if
29        len(chunk.page_content.strip()) > 40]
30
31    # Embedding and FAISS fase
32    embedding_model = OpenAIEmbeddings()
33    vector_store = FAISS.load_local(FAISS_INDEX, embedding_model,
34        allow_dangerous_deserialization=True)
35
36    # Prompt and query + Retrieval
37    question = "Nel 2024 sono state attivate 5 piattaforme
38        per il whistleblowing?"
39    retrieved_docs = vector_store.similarity_search(question)
40    docs_content = "\n\n".join(doc.page_content for doc in retrieved_docs)
41
42    # Generation
43    message = prompt.invoke({"question": question, "context": docs_content })
44    answer = llm_models["gpt-4.1"].invoke(message)

```

Listato 1: Script Python che illustra la fase di indexing e **Retrieval and Generation**.

Grazie a LangChain, il sistema RAG risulta facilmente estendibile ad altri modelli o fonti

di dati [19].

3.1.2 Analisi e preparazione dei dati

Questa sezione descrive la fase iniziale del progetto di un sistema RAG: la preparazione e l'analisi dei documenti. I dati forniti dall'azienda consistevano in conversazioni prelevate da un forum di DynDevice, fornite in formato CSV. Il primo passo è stato analizzare la struttura delle tabelle per comprendere il contenuto e individuare le informazioni più rilevanti. In seguito è stato avviato un processo di pulizia dei dati che ha portato alla rimozione di elementi ridondanti o privi di significato. Questa fase è cruciale perché la qualità e il corretto funzionamento del sistema RAG dipendono da una solida base di dati, condizione che garantisce anche una maggiore precisione nelle risposte fornite dal modello.

```
{
  "progress_id": 656,
  "subject": "AP - E-Learning MIM - SVILUPPO",
  "description": "E-Learning MIM",
  "created_at": "2003-10-20 20:22:33",
  "closed_at": "2003-11-28 20:22:33",
  "updated_at": "2003-11-28 13:38:40",
  "author": 9,
  "message_count": 2,
  "total_char_length": 2699,
  "distinct_authors": 1,
  "messages": [
    {
      "msg_id": 10506,
      "progress_id": 656,
      "answer_to_msg_id": 0.0,
      "content": "Questo progress specifica le caratteristiche della piattaforma e-learning di MIM.Obiettivo: creare un sistema automatico",
      "timestamp": "2003-10-21 11:12:26",
      "attachment": NaN,
      "author": 9
    },
    {
      "msg_id": 10944,
      "progress_id": 656,
      "answer_to_msg_id": 0.0,
      "content": "28.11.03 LM A AGI. Vorrei che facessi una analisi di quello che potremmo fare per lo streaming video dei nostri progetti",
      "timestamp": "2003-11-28 13:38:40",
      "attachment": NaN,
      "author": 9
    }
  ]
},
```

Figura 6: Formato JSON dei dati.

Come si vede dalla Figura 6 ogni conversazione è stata raggruppata per `progress_id` e presentata con diverse caratteristiche utili. Tra le altre si possono notare autore, data, descrizione, messaggio, lunghezza in caratteri ed eventuale presenza di allegati.

Parametro per ogni progress	Numero di messaggi	Lunghezza media dei messaggi	Lunghezza media in parole dei messaggi
Count	574.00	574.00	574.00
Mean	15.10	922.18	137.87
Std	63.44	1155.03	178.99
Min	1.00	22.00	4.00
25%	2.00	300.90	42.54
50%	4.00	584.75	85.64
75%	9.00	1136.73	164.88
Max	1153.00	13266.50	2036.00

Tabella 1: I parametri riportati indicano: *Count* (numero di valori non nulli), *Mean* (media aritmetica), *Std* (deviazione standard), *Min* (valore minimo), *25%*, *50%*, *75%* (quartili: 25%, mediana e 75%), e *Max* (valore massimo).

Terminata la pulizia dei dati, abbiamo ottenuto 574 progress con un totale di 8669 messaggi. Un’analisi più dettagliata è riportata in Tabella 1. Come si può notare, il numero medio di messaggi per progress è pari a 15, ma con una deviazione standard elevata (63.44), segno che alcuni progress contengono pochissimi messaggi mentre altri ne includono centinaia. La distribuzione è fortemente asimmetrica: il valore massimo raggiunge 1153 messaggi, mentre il 75% dei progress ne contiene al massimo 9. Un andamento analogo si riscontra nella lunghezza dei messaggi: la media è di circa 922 caratteri (137 parole), ma il massimo supera i 13.000 caratteri e i 2000 termini. Questi dati indicano che, sebbene la maggior parte delle conversazioni sia breve, esistono alcuni progress con messaggi molto lunghi e articolati, che possono influenzare il comportamento del sistema RAG e le prestazioni in fase di retrieval.

Sebbene la funzione **describe()** fornisca informazioni utili, i valori numerici da soli non sempre permettono di cogliere facilmente la distribuzione dei dati. Per questo motivo, è stata utilizzata anche una rappresentazione grafica mediante istogrammi (Figura 7 e Figura 8) che consentono di visualizzare la frequenza dei messaggi per ogni progress e la distribuzione delle lunghezze dei messaggi.

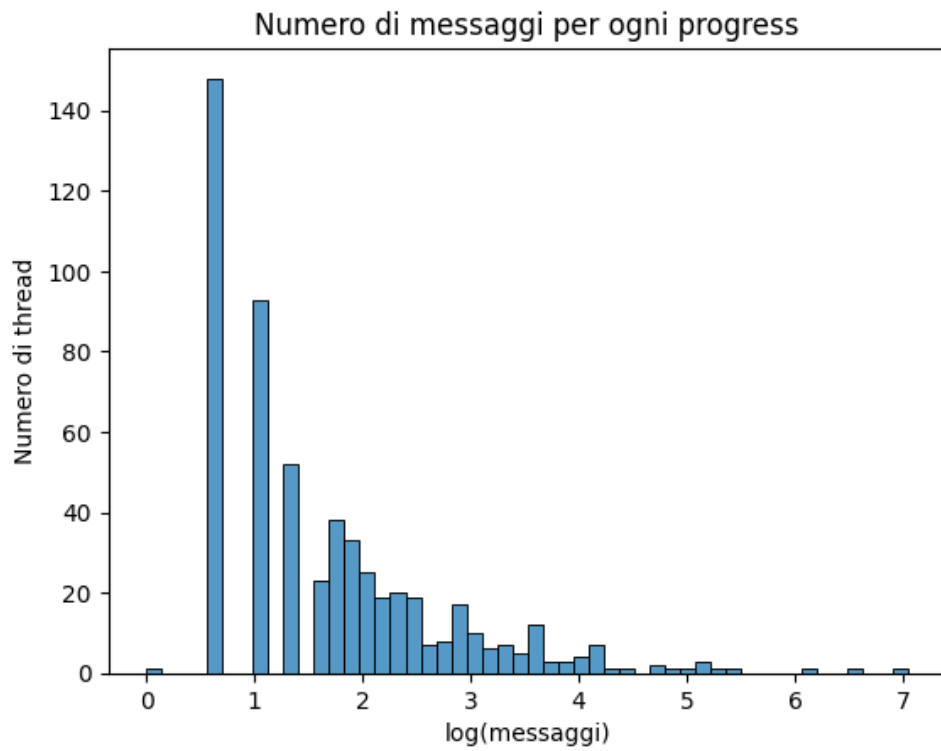


Figura 7: Distribuzione della lunghezza dei messaggi.

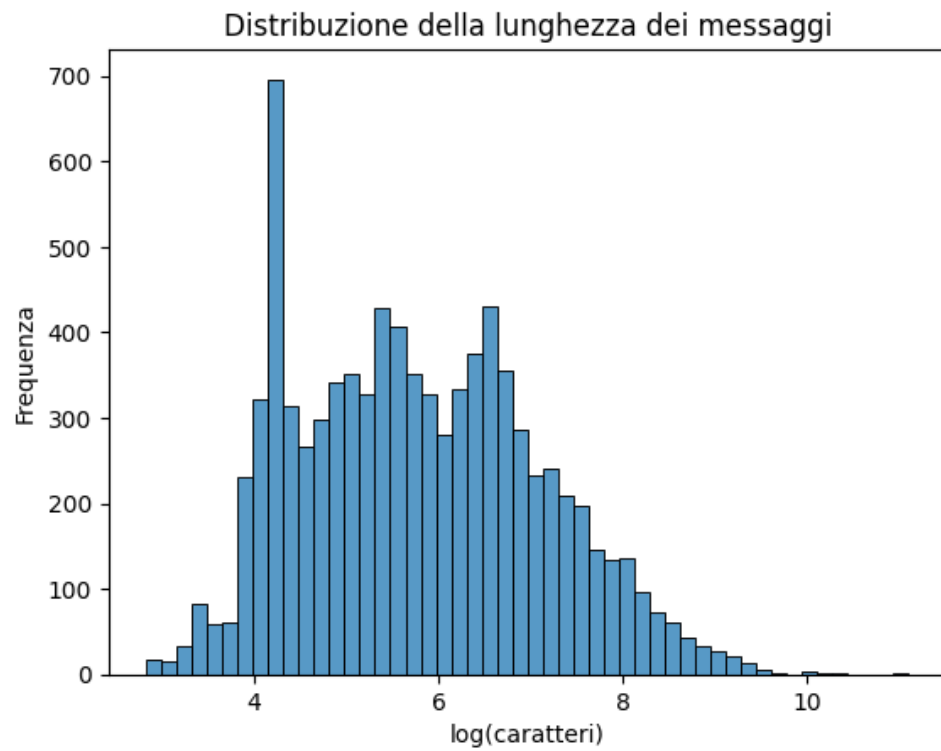


Figura 8: Numero di messaggi per ogni progress.

3.1.3 Splitting e Embedding

In questa sezione vengono svolte due azioni importanti: separazione dei documenti in chunk e conversione in vettori numerici seguita dal salvataggio in un database vettoriale. Le porzioni di testo più piccole vengono normalmente chiamate **chunk** e servono per rendere più comprensibile la ricerca delle informazioni da parte del modello LLM. Vi sono diverse tecniche di splitting che possono andare a separare porzioni di testo dai grandi documenti iniziali: splitting a livello semantico, di frase o per tokenizzazione. Splitting per tokenizzazione divide i testi con dimensioni fisse, dopo un certo numero di token, utile per controllare il limite massimo di input gestito dal modello. Splitting a livello di frase punta a preservare frasi o paragrafi migliorando la coerenza interna dei chunk e facilitando l'interpretazione semantica. Splitting a livello semantico suddivide il testo considerando la similarità semantica tra segmenti generando chunk omogenei e di senso compiuto. In questo progetto il processo di splitting è stato svolto grazie al **RecursiveCharacterTextSplitter** che significa dividere la grande porzione di testo per caratteri. Il passaggio successivo è molto sottile e influenza molto la precisione del sistema: bisogna decidere la dimensione e un eventuale sovrapposizione dei chunk. Questa scelta permette al modello di gestire meglio le unità senza perdere contesto. Normalmente la dimensione consigliata dei chunk varia tra i 256 e i 512 caratteri che garantisce un **average faithfulness** al 97,59% e un **average relevancy** al 97,78%. **Average faithfulness** indica se la risposta è fuorviante o fa matching con chunk e **average relevancy** se i chunk uniti alla risposta soddisfano la domanda dell'utente. Come altra scelta progettuale la sovrapposizione dei chunk permette ad ogni chunk successivo di condividere i caratteri con quello precedente, evitando di interrompere dei concetti. L'ultima scelta progettuale è stata quella di filtrare tra i chunk appena creati quelli che avevano lunghezza inferiore a 40, eliminando così chunk poco informativi e aumentando la qualità del retrieval [8].

La fase successiva riguarda l'**embedding** e il salvataggio nel database vettoriale, come illustrato in Figura 10. L'**embedding** consiste nel convertire ogni chunk in un vettore numerico, permettendo così al modello LLM di confrontare testi con funzioni di similarità. Nel progetto è stato utilizzato il modello di embedding di OpenAI che, tramite chiamate API, restituisce per ogni chunk un vettore.

Una chiamata API (Application Programming Interface) è una richiesta che il client invia a un servizio remoto, chiedendo accesso a dati o funzionalità. Le API costituiscono un insieme di definizioni e protocolli che permettono alle applicazioni di comunicare tra loro

[10]. Nel contesto del processo di embedding, il modello OpenAI riceve una chiamata API: al client viene richiesto di inviare ai server la porzione di testo (chunk) come input e dopo, tramite un protocollo definito (spesso HTTP), viene restituito come risposta il vettore numerico corrispondente [5].

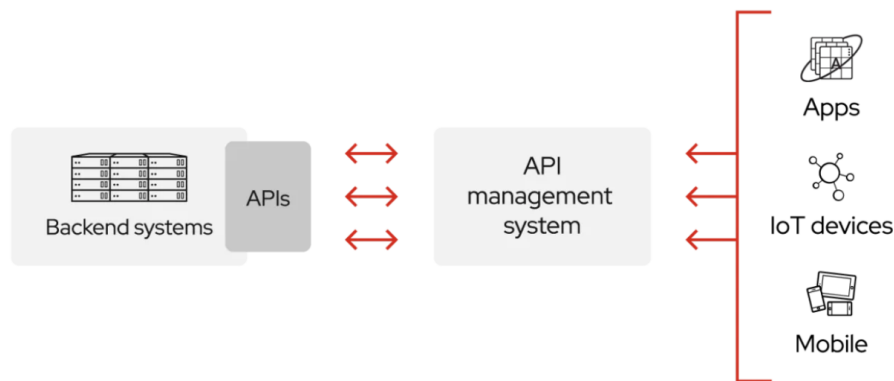


Figura 9: Chiamata API.

Come si può notare dalla Figura 9, l'API è rappresentata come un'interfaccia tra un'applicazione e il servizio. Il client invia una richiesta attraverso l'API e l'API la inoltra al sistema che esegue l'operazione, quindi la risposta viene restituita al client. Questo schema evidenzia il ruolo di mediatore svolto dall'API tra chi richiede un servizio e chi lo offre, sottolineando come il client non debba essere necessariamente a conoscenza dei dettagli interni.

```
1 message = prompt.invoke({"question": question, "context": doc_cont})
2 answer = llm_models["gpt-4.1"].invoke(message)
```

Listato 2: Script Python che svolge una chiamata API per generare la risposta.

Nel caso del progetto trattato, come illustrato nel Listato 2, si può notare la chiamata API che interessa la fase di generazione della risposta da parte dell'LLM (GPT-4.1) per rispondere alla query dell'utente.

Svolta l'operazione di embedding, viene creato un indice FAISS (**Facebook AI Similarity Search**), una libreria che serve per la ricerca vettoriale veloce. L'indice è capace di contenere tutti i vettori ed effettua ricerche basate sulla distanza tra i vettori, ad esempio per recuperare i chunk più pertinenti rispetto ad una query. Esistono altri tipi

di database vettoriali come Chroma, Milvus o Qdrant, ma FAISS si è rivelato essere il più semplice da integrare e sufficientemente performante per le esigenze del progetto. Al primo avvio vengono generate le embedding e l'indice FAISS salvato in locale così che per gli avvii successivi si possa evitare di ripetere il procedimento [8].

```
1 # Embedding and FAISS
2 embedding_model = OpenAIEmbeddings()
3 if os.path.exists(FAISS_INDEX):
4     print("Vector store gia' esistente. Carico quello.\n")
5     vector_store = FAISS.load_local(FAISS_INDEX, embedding_model,
6                                     allow_dangerous_deserialization=True)
7 else:
8     print("Vector store non trovato. Procedo con la creazione.\n")
9     vector_store = FAISS.from_documents(chunks, embedding_model)
10    # Saving the vector store
11    vector_store.save_local(FAISS_INDEX)
```

Listato 3: Script Python per l'implementazione del caricamento o della generazione di un indice vettoriale FAISS utilizzando embeddings OpenAI.

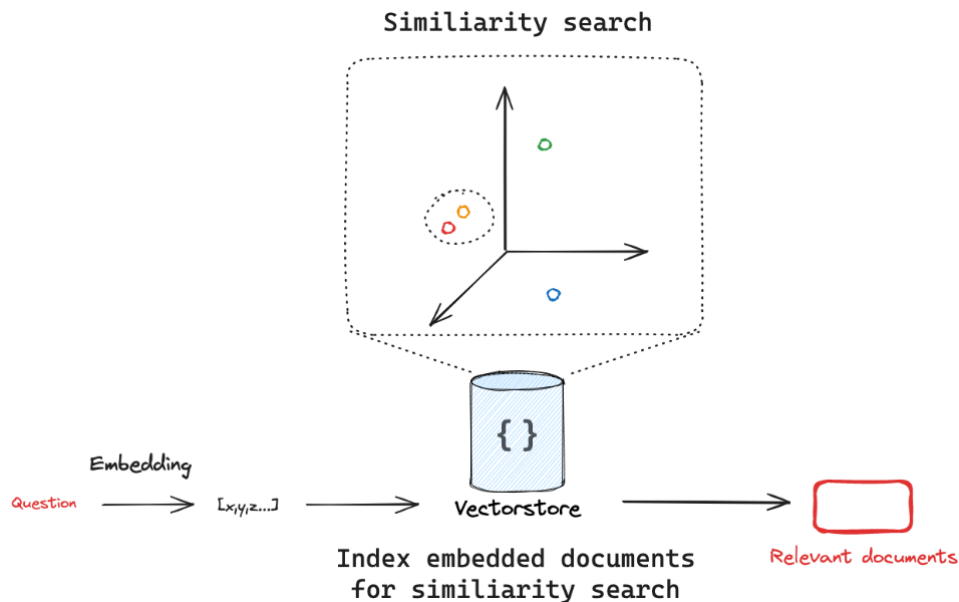


Figura 10: Il recupero dei chunk rilevanti a partire dalla query.

3.1.4 Prompt and query

Il prompt gioca un ruolo davvero importante per una corretta formulazione della risposta. Un prompt ben formulato non è solamente un **input**, ma una guida per ottenere una risposta rilevante e precisa. Una soluzione basata su prompt può essere molto incisiva anche in domini molto specialistici come ad esempio la medicina coreana [7]. È stato condotto un esperimento che ha confermato questi vantaggi, ottenendo migliori performance in rilevanza e informatività nonostante permangano alcune criticità come latenza e strutturazione del contenuto. Effettivamente con un prompt ben progettato si può essere efficaci e flessibili [7]. Nel progetto sono stati utilizzati due prompt distinti: il primo, denominato **boolean**, dedicato alle domande a risposta chiusa; il secondo, invece, per le domande aperte e richieste più complesse.

Prompt domande a risposta chiusa:

Rispondi alla seguente domanda in italiano con True o False, usando solo il contesto fornito. Rispondi comunque True o False, anche se non sei sicuro.

Contesto:

Domanda:

Risposta:

Prompt domande a risposta aperta:

Rispondi alla seguente domanda in italiano, utilizzando solo le informazioni fornite nel contesto. Se non sei sicuro, fornisci comunque la risposta più probabile in una frase breve.

Contesto:

Domanda:

Risposta:

Successiva alla creazione di un buon prompt vi è la metodologia di retrieval per recuperare le informazioni rilevanti per generare una risposta di qualità. Nel progetto sono stati utilizzati due differenti approcci: **Similarity Search** e **Max Marginal Relevance**. La prima punta direttamente alla rilevanza semantica ed è quello più utilizzato mentre

la seconda cerca di bilanciare rilevanza e diversità circoscrivendo il contesto ai top k risultati recuperati¹. Entrambi tecniche che puntano a ricercare e recuperare, per quanto possibile, i contesti più vicini alla query proposta dall'utente. La scelta della tecnica di retrieval incide direttamente sulla qualità della risposta prodotta: un contesto più pertinente e meno ridondante permette al modello di formulare risposte più accurate. Come evidenziato in studi recenti, approcci di reranking possono migliorare ulteriormente la precisione, ma comportano un aumento della complessità computazionale e della latenza [8].

Oltre a questi metodi di retrieval, la ricerca recente sta esplorando anche approcci **ibridi** che combinano tecniche differenti per migliorare ulteriormente la qualità e la coerenza del contesto fornito al modello. Un esempio di metodologia ibrida mediante utilizzo dei grafi di conoscenza è K^2RAG , come illustrato in Figura 11. Questo modello adotta un paradigma divide et impera per combinare ricerca vettoriale, grafi di conoscenza e sintesi testuale. I documenti vengono prima riassunti, poi indicizzati sia in forma vettoriale sia in knowledge graph² per migliorare la qualità di retrieval, la coerenza delle risposte e ridurre i tempi di addestramento, l'overhead computazionale e il consumo di VRAM³ [9].

¹Per *top-k* si intendono i k chunk più rilevanti restituiti dalla ricerca, ordinati in base a un punteggio di similarità. L'approccio **Max Marginal Relevance** cerca di massimizzare la rilevanza riducendo la ridondanza tra questi risultati.

²Un knowledge graph è una struttura dati che rappresenta informazioni sotto forma di nodi (entità) e archi (relazioni), utile per organizzare e collegare conoscenza in modo semantico.

³Per VRAM si intende Video RAM, la memoria della scheda grafica (GPU memory)

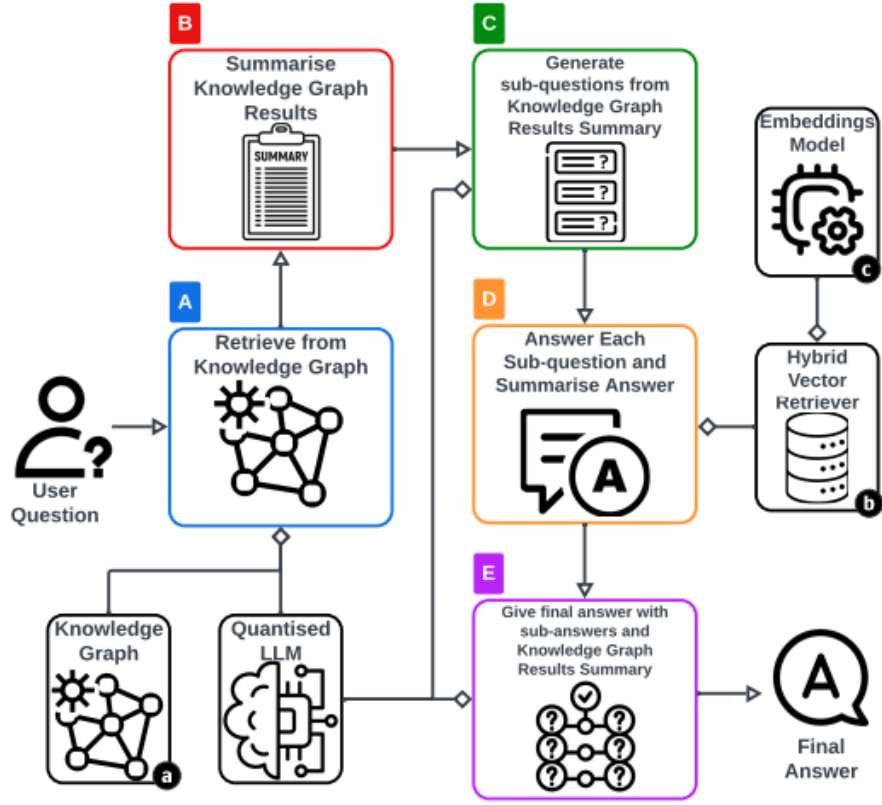


Figura 11: Panoramica del framework K^2RAG . Componente (a): Knowledge Graph. Componente (b): Hybrid Retriever contenente Dense e Sparse retrievers. Componente (c): Modello di Embeddings. Passo (A): risultati dal Knowledge Graph. Passo (B): sintesi dei risultati del Knowledge Graph. Passo (C): generazione di sotto-domande dai chunks. Passo (D): generazione di sotto-risposte per le sotto-domande. Passo (E): generazione della risposta finale a partire dalle sotto-risposte e dal riepilogo dei risultati del Knowledge Graph.

Una volta ottenuto il contesto rilevante tramite il processo di **retrieval**, il passo successivo consiste nell'invocare il modello LLM per generare la risposta. Questo processo avviene concatenando la domanda dell'utente con il contesto recuperato in un unico **prompt**. Questa fase implica due chiamate API distinte: la prima al **Vector Store FAISS** per l'esecuzione della ricerca vettoriale, la seconda all'API del modello LLM per la generazione della risposta finale. La qualità del contesto e la struttura del prompt influenzano direttamente la pertinenza, la precisione e la coerenza della risposta, come già detto in precedenza. Nel progetto, ad esempio, alla query "Nel 2024 sono state attivate 5 piattaforme per il whistleblowing?" sono stati recuperati i documenti semanticamente più vicini, concatenati nel prompt e inviati al modello LLM, ottenendo

così la risposta corretta.

Nel capitolo successivo verranno presentate le tipologie di query utilizzate per testare il sistema, comprendenti sia domande a risposta chiusa sia domande a risposta aperta. Tali richieste, formulate su diversi domini e livelli di difficoltà, hanno permesso di valutare le prestazioni dei modelli.

4 Analisi Sperimentale

In questo capitolo vengono presentati i test condotti sul sistema RAG sviluppato, con l'obiettivo di valutare le prestazioni. Dopo aver descritto nel capitolo precedente le fasi di progettazione e implementazione, qui verranno illustrati il protocollo di sperimentazione, le tipologie di query utilizzate e i criteri di valutazione adottati. I risultati ottenuti saranno quindi discussi, evidenziando punti di forza, limiti e possibili miglioramenti.

4.1 Modalità di test

La fase sperimentale ha l'obiettivo di andare a valutare l'accuratezza di 6 modelli - 4 di OpenAI, uno di Google e uno di Mistral AI - nel rispondere ad un insieme di 65 domande suddivise in due categorie. Per condurre questa analisi sono stati utilizzati due approcci:

- Il primo test utilizza una metodologia standard di retrieval basata su **Similarity Search**;
- Il secondo test utilizza una metodologia **Max Marginal Relevance** con reranking dei documenti.

Nella sperimentazione del secondo test, la strategia adottata puntava al miglioramento della pertinenza dei documenti recuperati, ma questo ha portato a dei rallentamenti nella generazione delle risposte da parte del sistema. In entrambi i test, la prima parte riguarda 45 domande a risposta chiusa, mentre la seconda riguarda le rimanenti 20 domande a risposta aperta, maggiormente complesse. Il criterio di valutazione adottato è quello dell'accuracy, la percentuale di risposte esatte. Questo permette di evidenziare in maniera oggettiva le prestazioni dei modelli, mettendo in evidenza eventuali differenze, in particolare nei risultati ottenuti dalle domande aperte. Come verrà illustrato in seguito, per le domande aperte è stata introdotta una modalità di valutazione per similarità, tramite la funzione di cosine similarity, mediante utilizzo di un parametro di soglia detto **threshold**.

4.1.1 Domande e risposte

Il dataset utilizzato per la sperimentazione è costituito da 65 domande, suddivise in due categorie: domande a risposta chiusa (45) e domande a risposta aperta (20). Le prime presentano una risposta vero, falso o non lo so mentre le seconde richiedono risposte più articolate consentendo di valutare le capacità dei modelli. Le domande sono state

formulate sulla base dei contenuti presenti nel dataset aziendale e coprono differenti argomenti e livelli di difficoltà, al fine di testare in maniera più completa le prestazioni del sistema. In Tabella 2 sono riportati alcuni esempi rappresentativi per entrambe le tipologie.

Domanda	Tipologia	Risposta attesa
Il riesame si basa anche sulle informazioni provenienti dai clienti?	Chiusa	Vero
Nella maggior parte delle settimane analizzate, i picchi negativi di accessi da newsletter si verificano di venerdì?	Chiusa	Falso
A cosa serve il progress “Qualità - nuove idee”?	Aperta	Serve a raccogliere suggerimenti e proposte da parte degli utenti MIM per migliorare i processi qualitativi, con uno spazio aperto all’innovazione interna.

Tabella 2: Esempi di domande chiuse e aperte con le rispettive risposte attese.

Le domande e le risposte sono state caricate nel sistema RAG: durante i test, ogni domanda è stata unita ai *chunk* recuperati dal sistema e inserita nel *prompt* del modello, al fine di confrontare la risposta generata con quella attesa.

4.1.2 Metodologia Similarity Search

In questo test la prima parte affronta le 45 domande a risposta chiusa utilizzando la tecnica di retrieval di **Similarity Search**, che permette di rilevare la somiglianza semantica tra i chunk recuperati e aiutare nella generazione di una risposta precisa. Questo approccio, pur essendo meno sofisticato di metodologie ibride o integrate con grafi della conoscenza, si è rivelato efficace per il dataset di dimensioni limitate, garantendo tempi di risposta rapidi e una buona pertinenza delle informazioni recuperate. La ricerca si basa sul calcolo della similarità tra la query e i documenti indicizzati, permettendo di identificare i chunk più vicini semanticamente e utilizzarli nella generazione della risposta da parte del modello.

In particolare possiamo notare come la valutazione delle domande chiuse è stata svolta attraverso una semplice funzione `compare_answers(generated, correct)` e utilizzata in seguito per calcolare la percentuale di risposte esatte.

```

1 # Comparison function
2 def compare_answers(generated, correct):
3     return generated.strip().lower() == correct.strip().lower()
4
5 # % of success
6 def calculate_success_rate(generated_answers, correct_answers):
7     assert len(generated_answers) == len(correct_answers), "Numero di
risposte deve coincidere"
8     matches = sum(compare_answers(gen, cor) for gen, cor in zip(
generated_answers, correct_answers))
9     return matches / len(correct_answers) * 100

```

Listato 4: Script Python per il calcolo dell'exact match.

Per le restanti 20 domande a risposta aperta, è stata introdotta la **Cosine Similarity**, calcolata:

$$\text{cosine_sim}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|},$$

dove A e B sono i vettori di embedding generati dal modello. Questa funzione assume valori da -1, vettori opposti, a +1, per i vettori identici. Questa metrica è ampiamente utilizzata in applicazioni NLP perché è robusta alla lunghezza del vettore e mette in relazione la direzione dei vettori [6, 14]. Nel sistema RAG progettato è stata creata un'apposita funzione **semantic_similarity(answer1, answer2)** che restituisce la **Cosine Similarity** di due vettori, del pacchetto `sklearn.metrics.pairwise`.

```

1 from sklearn.metrics.pairwise import cosine_similarity
2
3 # Semantic similarity function
4 def semantic_similarity(answer1, answer2):
5     vec1 = embedding_model.embed_query(answer1)
6     vec2 = embedding_model.embed_query(answer2)
7     return cosine_similarity([vec1], [vec2])[0][0]

```

Listato 5: Script Python.

Il fattore più importante rimane il metodo di confronto utilizzato per determinare la correttezza booleana delle risposte alle domande aperte: una volta calcolata la **Cosine Similarity**, viene impostato il livello di soglia **threshold** che permette di verificare la correttezza della risposta proposta dal modello LLM. Nel caso del sistema RAG in oggetto sono state verificate tre soglie di correttezza: 0.5, 0.75 e 0.9. Poiché la **Cosine**

Similarity assume valori compresi tra -1 e 1 , ma negli embedding testuali reali i valori si distribuiscono quasi sempre tra 0 e 1 , le soglie adottate rappresentano livelli crescenti di corrispondenza semantica. Una soglia bassa (0.5) consente di accettare risposte che condividono solo una parte del significato con quella attesa, risultando più tollerante. Una soglia media (0.75) richiede una corrispondenza semantica più stretta, mentre una soglia alta (0.9) è molto restrittiva e accetta solo risposte quasi identiche dal punto di vista semantico. Questo approccio è particolarmente rilevante per le domande aperte, dove l'espressione testuale può variare notevolmente pur trasmettendo lo stesso concetto.

4.1.3 Metodologia Max Marginal Relevance

In questo test la prima parte affronta le 45 domande a risposta chiusa utilizzando la tecnica di **Max Marginal Relevance (MMR)** che cerca di equilibrare rilevanza e diversità nei documenti recuperati. Questo è importante in un contesto RAG poiché, riducendo la ridondanza, si migliora l'efficienza del sistema. Dopo il primo retrieval basato su **Similarity Search**, è stato applicato un reranking dei chunk mediante MMR, in modo da promuovere documenti semanticamente nuovi e utili per la risposta finale. Nel Listato 6 possiamo notare la fase di retrieval e reranking dei documenti.

```
1 ret_docs = vector_store.max_marginal_relevance_search(question, k=6)
2 final_docs = rerank(question, retrieved_docs, top_n=3)
```

Listato 6: Script Python: MMR e reranking dei documenti.

Per le restanti 20 domande a risposta aperta, il metodo di valutazione rimane invariato: la differenza tra i due approcci riguarda esclusivamente il modo in cui vengono recuperati i chunk.

4.2 Analisi dei risultati

In questa fase andiamo ad analizzare i risultati ottenuti dai test effettuati. Visualizziamo una tabella, alcuni grafici e confrontiamo le varie metodologie in base alle prestazioni e all'accuratezza delle risposte. La Tabella 3 riporta tutti i risultati dei test svolti: Test1, Test2 e Test3 riguardano 3 set differenti di domande a risposta chiusa, mentre Test4 e Test5 riguardano 2 set differenti di domande a risposta aperta. Per i modelli utilizzati,

notiamo come **GPT-4-turbo** sia molto efficace ottenendo le percentuali più alte, 87%, mentre **Mistral-Tiny** mostra una maggiore variabilità, 69%. Inoltre, notiamo come per entrambi i test, con **threshold** pari a 0.5 e 0.75, i risultati con metodologia standard sono tutti ottimi, mentre per **threshold** pari a 0.9 la situazione si fa più delicata e variabile. L'utilizzo della metodologia **Max Marginal Relevance** subisce un calo di accuratezza nelle risposte con percentuali più basse (per esempio **GPT-4-turbo** per il Test1 ha ottenuto il 26.67%), a dimostrazione che questa metodologia con reranking non offre alcun beneficio.

Retrieval strategy	Modello LLM	% successo (chiuse)			% successo (aperte)					
		Test1	Test2	Test3	Test4			Test5		
					0.50	0.75	0.90	0.50	0.75	0.90
Similarity Search	GPT-4.1	73.33	86.67	73.33	100	100	40	100	100	10
	GPT-4-turbo	86.67	86.67	86.67	100	100	50	100	100	10
	GPT-4	86.67	86.67	80	100	100	50	100	100	10
	GPT-3.5-turbo	86.67	66.67	80	100	100	60	100	100	10
	Gemini-2.5-flash	73.33	86.67	60	100	100	60	100	100	10
	Mistral-tiny	73.33	73.33	60	100	100	70	100	100	20
MMR	GPT-4.1	53.33	60	46.67	100	70	30	100	70	10
	GPT-4-turbo	26.67	40	6.67	100	10	0	100	20	10
	GPT-4	26.67	40	6.67	100	10	0	100	20	10
	GPT-3.5-turbo	80	66.67	66.67	100	100	30	100	90	10
	Gemini-2.5-flash	33.33	40	20	100	60	30	100	40	10
	Mistral-tiny	66.67	53.33	60	100	100	50	100	100	10

Tabella 3: Risultati per strategie di retrieval e modelli LLM

Come possiamo osservare dalla Tabella 3, la metodologia che ha riscontrato le maggiori percentuali di successo è quella standard, strategia di retrieval per **Similarity Search**. A partire dalle domande a risposta chiusa notiamo già una gran differenza di percentuali di successo in quanto la metodologia di reranking e di **Max Marginal Relevance** non è molto efficace. Questo indica come il contesto che viene recuperato e utilizzato con il metodo standard è già molto buono mentre, con la metodologia di **Max Marginal Relevance**, il reranking non porta a risultati migliori, come ipoteticamente ci si poteva aspettare. Non solo nelle domande a risposta chiusa, ma anche nelle domande a risposta aperta possiamo notare questa differenza: le percentuali di successo riscontrate con la metodologia standard con un threshold inferiore a 0.9 risultano essere sempre del 100%, seguendo la verifica di correttezza per **Cosine Similarity**. Nella metodologia **Max Marginal Relevance** notiamo che già con threshold 0.75 alcuni modelli, come Gemini-2.5-flash o GPT-4, riscontrano percentuali di successo inferiori. Il livello di soglia 0.9 è molto particolare perché richiede che le risposte ottenute dai modelli LLM siano

praticamente identiche a quelle attese (il livello massimo a cui si può arrivare è 1), ma sappiamo benissimo che è quasi impossibile. Per questo motivo notiamo che tra le due metodologie non ci sono molte differenze nel controllo delle risposte aperte con soglia maggiore o uguale di 0.9. Tra tutti i modelli spicca Mistral-Tiny che si comporta sorprendentemente bene nei Test4 e Test5 con threshold pari a 0.9. In sintesi, i risultati dimostrano come la metodologia standard sia più efficace: il reranking con MMR, pur teoricamente promettente, non ha migliorato la pertinenza delle risposte e in alcuni casi ha addirittura compromesso le performance.

Da questo grafico, Figura 12, si può visualizzare più facilmente la percentuale di successo dei modelli per le due metodologie proposte.

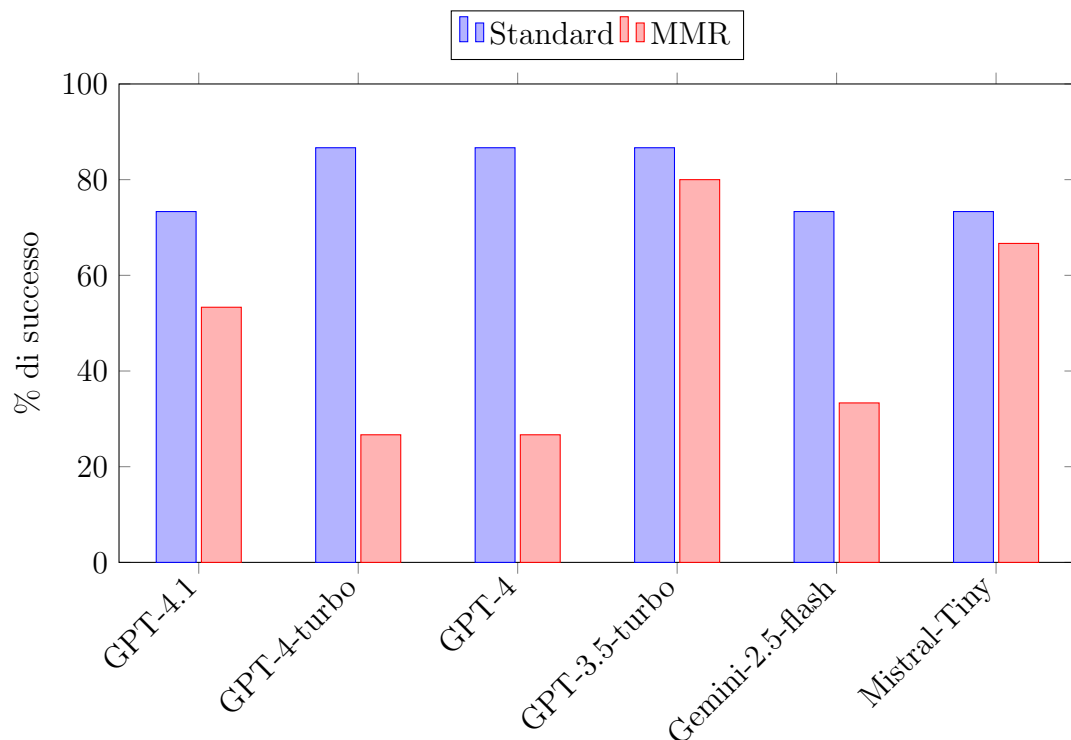


Figura 12: Confronto delle percentuali di successo per Test1 con metodologia Standard e MMR.

Dopo aver analizzato i risultati ottenuti con l'architettura attuale basata su **Retrieval and Generation**, è interessante confrontarli con l'architettura precedente priva di RAG. Quest'ultima utilizzava la stessa base documentale, ma la fase di generazione delle risposte si basava su riassunti dell'intera documentazione, adottando il metodo **IDEA**:

i progress venivano organizzati in gruppi, producendo un riassunto per ciascuno e unendoli successivamente mantenendo l'ordine cronologico. Dal grafico in Figura 13 emerge chiaramente come i modelli presi in esame ottengano risultati migliori con l'approccio RAG rispetto all'architettura precedente basata solo su IDEA. In particolare, mentre l'architettura precedente fornisce già prestazioni discrete, l'uso di RAG permette di ottenere risultati significativamente più accurati e robusti, soprattutto per modelli come GPT-4 (87%) e Mistral-Tiny (73%); solo Gemini-2.5-flash mostra prestazioni simili nei due scenari, circa 87%.

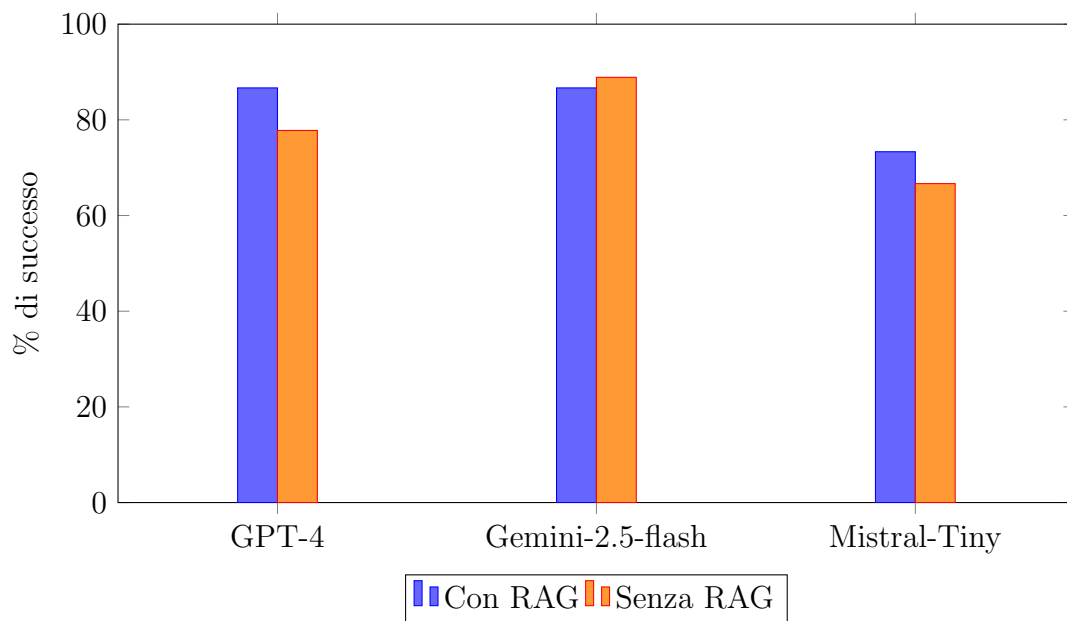


Figura 13: Confronto delle percentuali di successo per Test2 tra progetto trattato in questa tesi e progetto precedente.

4.2.1 Confronto dei modelli LLM

Oltre a confrontare le diverse metodologie di retrieval, è utile considerare le differenze di prestazioni tra i modelli LLM utilizzati all'interno del sistema RAG. Dall'analisi complessiva dei risultati emergono alcune considerazioni:

- **Modelli GPT-4 (GPT-4.1, GPT-4-turbo, GPT-4):** in generale hanno garantito le prestazioni più stabili e consistenti, sia con la metodologia standard sia con MMR. In particolare **GPT-4.1** e **GPT-4-turbo** hanno mostrato i valori medi di successo più elevati nelle domande a risposta chiusa (86%), mantenendo un buon

livello anche con soglie di similarità più restrittive (30-35%). Questo suggerisce che siano i modelli più robusti e affidabili nel recupero delle informazioni;

- **GPT-3.5-turbo**: ha fornito prestazioni discrete, con percentuali di successo buone in alcuni test (87%) ma meno costanti rispetto alle versioni più avanzate della famiglia GPT-4. Risulta un compromesso accettabile tra accuratezza e costi computazionali;
- **Gemini-2.5-flash**: ha evidenziato risultati variabili. In alcuni test si è avvicinato alle performance di GPT (87%), mentre in altri è stato significativamente inferiore (60%). Tra tutti i modelli utilizzati forse quello meno convincente;
- **Mistral-Tiny**: sebbene in alcuni casi abbia ottenuto percentuali paragonabili a quelle di Gemini, ha mostrato forti oscillazioni.



Figura 14: Confronto delle percentuali di successo per domande chiuse con metodologia Similarity Search e MMR.

Nel Grafico 14 è riportato il confronto tra i modelli LLM utilizzati per le domande a risposta chiusa, mostrando la media delle percentuali di successo ottenute dal dataset di

domande dei Test1, Test2 e Test3. Si osserva chiaramente come la metodologia Similarity Search risulti superiore rispetto a MMR, nonostante Mistral-Tiny o GPT-3.5-turbo si comportino quasi allo stesso modo. In quest'altro Grafico 15, è riportato il confronto tra i modelli LLM per le domande a risposta aperta, mostrando la media delle percentuali di successo ottenute per i diversi valori di threshold dal dataset di domande dei Test4 e Test5. Anche in questo caso notiamo come i modelli GPT-4.1, GPT-3.5-turbo e Mistral-Tiny presentano prestazioni relativamente simili tra le due tecniche.

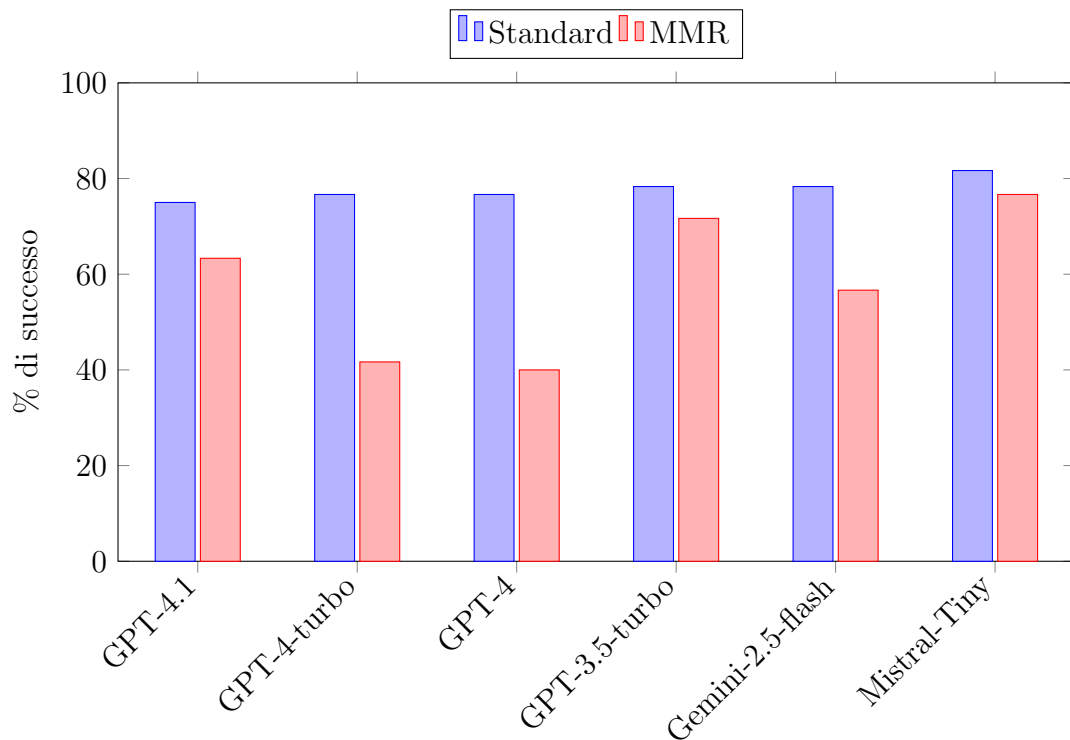


Figura 15: Confronto delle percentuali di successo per domande aperte con metodologia Similarity Search e MMR.

In sintesi, i risultati indicano che i modelli della famiglia **GPT-4** rappresentano la scelta ottimale per il sistema RAG sviluppato, grazie alla maggiore robustezza e alla capacità di fornire risposte più accurate e consistenti. Tra questi, **GPT-4** e **GPT-4-turbo** si distinguono come i modelli migliori, mentre **GPT-4.1** e **GPT-3.5-turbo** costituiscono alternative meno performanti ma comunque utilizzabili. I modelli **Gemini-2.5-flash** e **Mistral-Tiny**, pur mostrando alcune buone performance, non garantiscono la stessa stabilità e risultano quindi meno adatti.

5 Conclusione

L'obiettivo di questa tesi è stato lo sviluppo e la valutazione di un sistema Retrieval-Augmented Generation (RAG) applicato a un contesto documentale specifico. Dopo una fase iniziale di analisi, è stata realizzata l'architettura del sistema, con particolare attenzione alla fase di retrieval e alla successiva generazione delle risposte.

Il sistema è stato testato con un dataset costituito da 65 domande suddivise in due categorie: 45 a risposta chiusa e 20 a risposta aperta. Sono state implementate due strategie di retrieval: la metodologia standard basata su **Similarity Search** e un approccio alternativo con **Max Marginal Relevance (MMR)**. Le risposte sono state valutate con metodi diversi: **accuracy** per le domande chiuse e **Cosine Similarity** con **threshold** per quelle aperte.

I risultati ottenuti da questa sperimentazione mostrano come l'approccio al sistema RAG si sia rivelato efficace e possa aiutare l'utente a trovare informazioni utili in modo più semplice e veloce. Facendo questo si eviterebbero lunghe ricerche e l'assistente virtuale potrebbe fornire in pochi istanti la risposta alla domanda che l'utente si pone. In questo senso l'assistente intelligente è stato capace di interpretare correttamente i documenti di partenza e di trovare, per la maggior parte delle volte, del contesto rilevante per fornire una risposta accurata e precisa.

Tra le due metodologie sperimentate, quella standard per **Similarity Search** si è rivelata essere la più affidabile, garantendo in maniera consistente dei risultati ottimali. Ciò non implica che l'approccio con **Max Marginal Relevance** debba essere considerato meno valido in assoluto: in contesti differenti potrebbe offrire dei risultati competitivi con la prima metodologia. Discorso analogo per i modelli LLM utilizzati: nonostante per questa trattazione la famiglia di **GPT-4** ha garantito le prestazioni migliori in tutte le tipologie di domanda, l'evoluzione continua di questi modelli apre la strada a sperimentazioni ancora più avanzate.

In tale prospettiva, emergono alcuni possibili miglioramenti: si potrebbe ampliare la documentazione di partenza, associare ai chunk metadati come autore e data, oppure mostrare direttamente nelle risposte generate gli autori di riferimento. Un ulteriore sviluppo riguarderebbe l'integrazione di link e allegati presenti nel forum DynDevice, così da arricchire ulteriormente il dataset e migliorare la precisione delle risposte fornite dal modello. Anche la fase di splitting dei documenti potrebbe essere perfezionata, includendo metadati utili che il sistema potrebbe successivamente citare o rendere disponibili nel

contesto recuperato in fase di retrieval. Tali implementazioni contribuirebbero a rendere l'assistente virtuale uno strumento sempre più affidabile ed efficace per la formazione digitale, in grado di supportare in maniera solida le esigenze degli utenti.

In conclusione, il lavoro svolto dimostra come l'integrazione tra **Retrieval & Generation** possa rappresentare un passo significativo verso la realizzazione di sistemi di supporto intelligenti, capaci di migliorare l'accesso e la fruizione delle informazioni in ambito formativo.

Riferimenti bibliografici

- [1] Ashish Vaswani et al. *Attention Is All You Need*. arXiv preprint arXiv:1706.03762. 2023. URL: <https://arxiv.org/pdf/1706.03762>.
- [2] Amazon Web Services. *What is Retrieval-Augmented Generation*. <https://aws.amazon.com/what-is/retrieval-augmented-generation/>. 2024.
- [3] Yunfan Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv preprint arXiv:2312.10997. 2024. URL: <https://arxiv.org/pdf/2312.10997>.
- [4] Shailja Gupta, Rajesh Ranjan e Surya Narayan Singh. *A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions*. arXiv preprint arXiv:2410.12837. 2024. URL: <https://arxiv.org/abs/2410.12837>.
- [5] IBM. *Cos'è un'API (application programming interface)?* <https://www.ibm.com/it-it/think/topics/api>. 2024.
- [6] Kush Juvekar e Anupam Purwar. *COS-Mix: Cosine Similarity and Distance Fusion for Improved Information Retrieval*. arXiv preprint arXiv:2406.00638. 2024. URL: <https://arxiv.org/pdf/2406.00638>.
- [7] Bongsu Kang et al. *Prompt-RAG: Pioneering Vector Embedding-Free Retrieval-Augmented Generation in Niche Domains, Exemplified by Korean Medicine*. arXiv preprint arXiv:2401.11246. 2024. URL: <https://arxiv.org/pdf/2401.11246>.
- [8] Xiaohua Wang et al. *Searching for Best Practices in Retrieval-Augmented Generation*. arXiv preprint arXiv:2407.01219. 2024. URL: <https://arxiv.org/pdf/2407.01219>.
- [9] Hruday Markondapatnaikuni et al. *KeyKnowledgeRAG (K^2 RAG) : AnEnhancedRAGmethodfor answeringcapabilities*. arXiv preprint arXiv:2507.07695. 2025. URL: <https://arxiv.org/pdf/2507.07695>.
- [10] Red Hat. *Cosa sono le API (Application Programming Interface)*. <https://www.redhat.com/it/topics/api/what-are-application-programming-interfaces>. 2025.
- [11] SAS Institute Inc. *Le cinque tecnologie alla base dell'Intelligenza Artificiale (AI)*. https://www.sas.com/it_it/insights/articles/analytics/five-ai-technologies.html. 2025.

- [12] Wikipedia. *Apprendimento automatico, Machine Learning*. https://it.wikipedia.org/wiki/Apprendimento_automatico. 2025.
- [13] Wikipedia. *Apprendimento profondo, Deep Learning*. https://it.wikipedia.org/wiki/Apprendimento_profondo. 2025.
- [14] Wikipedia. *Cosine similarity*. https://en.wikipedia.org/wiki/Cosine_similarity. 2025.
- [15] Wikipedia. *Modello linguistico di grandi dimensioni*. https://it.wikipedia.org/wiki/Modello_linguistico_di_grandi_dimensioni. 2025.
- [16] Amazon Web Services. *Che cos'è un LLM (Large Language Model)?* <https://aws.amazon.com/it/what-is/large-language-model/>.
- [17] HuggingFace. *Come funzionano i Transformer?* URL: <https://huggingface.co/learn/llm-course/it/chapter1/4?fw=pt>.
- [18] IBM. *Cosa sono gli LLM?* <https://www.ibm.com/it-it/think/topics/large-language-models>.
- [19] LangChain. *Build a Retrieval Augmented Generation (RAG) App*. <https://python.langchain.com/docs/tutorials/rag/#overview>.